

信息安全理论与技术实验报告

实验一：基于容器化微服务与电影相似度推荐的系统实现

姓 名：程浩

学 号：241020055

学 院：计算机与大数据学院

专 业：信息安全

年 级：2024 级

指导教师：邹剑

基于容器化微服务与电影相似度推荐的系统实现

摘要

随着互联网应用和数字娱乐平台的快速发展，用户面对的电影、短视频、音乐和商品等信息资源不断增长，信息过载问题日益明显。推荐系统能够根据用户历史行为和物品属性，从大量候选内容中筛选出更可能符合用户兴趣的结果，是现代 Web 应用和数据智能系统中的重要基础能力。本课程设计以电影推荐场景为对象，基于 MovieLens 数据集构建一个可运行、可交互、可部署和可初步评估的电影推荐系统原型。

系统采用 FastAPI 提供后端接口，使用 Streamlit 实现前端交互页面，使用 PostgreSQL 保存电影、用户、评分、点击事件和实验结果等数据，使用 Redis 缓存推荐结果，并通过 Docker Compose 完成多服务统一编排。在推荐方法上，系统以电影到电影的相似推荐为主流程，基于用户评分行为构造采样评分向量并计算余弦相似度，返回 Top-K 相似电影及推荐解释。系统还提供电影检索、推荐点击统计、冷启动电影添加、健康检查、缓存指标和抽样离线评估等功能。

实验部分以 MovieLens 32M 级别数据为主要数据基础，数据库中包含 200,948 个用户、87,618 部电影和 32,921,437 条评分记录。测试结果表明，系统能够完成电影搜索、相似电影推荐、推荐解释和评估指标展示等核心流程；同时，抽样评估结果也反映出当前在线相似度计算在大规模数据下仍存在响应延迟偏高、热门电影偏置和个性化能力有限等问题。后续可通过离线预计算、ALS/BPR 模型训练、RecBole 评估和 Spark ALS 对照实验进一步完善系统。

关键词：电影推荐；MovieLens；FastAPI；PostgreSQL；Redis；Docker Compose

目 录

摘要	I
1 研究背景与课题意义	1
1.1 研究背景	1
1.2 课题意义	1
1.3 课程设计目标	2
2 需求分析	3
2.1 需求概述	3
2.2 用户角色与使用场景	3
2.3 功能需求	3
2.4 数据需求	4
2.5 非功能需求	5
2.6 开发与运行环境	5
3 总体设计	6
3.1 系统总体架构	6
3.2 系统模块划分	6
3.3 数据集与数据库设计	6
4 详细设计与系统实现	8
4.1 电影相似推荐设计	8
4.2 接口与缓存设计	8
4.3 点击率与冷启动设计	8
4.4 系统实现流程	9
4.5 功能测试方案	9
4.6 离线评估方案	9
5 测试结果与分析	10
5.1 功能测试结果	10
5.2 抽样评估结果	10
5.3 结果分析	10
6 存在问题、改进方向与总结	11
6.1 热门电影偏置	11
6.2 响应延迟	11
6.3 实验完整性	11
6.4 课程设计总结	11
参考文献	13

研究背景与课题意义

1.1 研究背景

随着互联网应用和数字娱乐平台的快速发展,电影、短视频、音乐、新闻和商品等信息资源呈现爆炸式增长。用户在获得更丰富选择的同时,也面临越来越明显的信息过载问题。以在线电影平台为例,平台中通常包含大量不同年代、类型、国家和风格的影片,如果只依靠用户主动搜索、人工分类或热门排行榜,用户很难在有限时间内快速找到真正符合个人兴趣的内容。与此同时,平台积累了大量用户评分、浏览、点击和收藏等行为数据,这些数据中包含了用户偏好和群体兴趣之间的关联。如果不能对这些数据进行有效建模和利用,就会造成数据价值的浪费。推荐系统研究表明,用户-物品交互数据能够有效支持个性化信息过滤与内容发现^[1]。

推荐系统正是在这样的背景下产生并不断发展的。推荐系统通过分析用户历史行为、物品属性以及用户与物品之间的交互关系,从大量候选内容中筛选出更可能满足用户需求的结果。与传统搜索系统相比,推荐系统不要求用户明确输入完整需求,而是能够根据历史偏好主动提供个性化内容;与简单排行榜相比,推荐系统又能够考虑不同用户之间的兴趣差异,从而提高内容发现效率。因此,推荐系统已经成为现代 Web 应用、电子商务、在线视频平台和数据智能系统中的重要基础能力。工业界典型系统如 Amazon 的 item-to-item 协同过滤也说明,相似物品推荐可以在大规模在线服务中取得较好的实用效果^[2]。

电影推荐是推荐系统研究和教学中非常典型的应用场景。一方面,电影具有标题、类型、评分、标签等较清晰的数据结构,便于构建可解释的推荐流程;另一方面,用户对电影的评分和观看行为能够较好反映兴趣偏好,适合用于协同过滤、相似度计算、矩阵分解和离线评估等实验。MovieLens 数据集作为推荐系统领域常用的公开数据集,提供了稳定的电影评分数据和较规范的数据格式,既适合算法实验,也适合作为课程设计的数据基础^[3]。在算法方面,基于物品的协同过滤、矩阵分解、隐式反馈建模和排序优化等方法均是电影推荐实验中常见的技术路线^[4-7]。

在实际应用中,推荐系统不仅是一个算法问题,也是一个完整的软件工程问题。一个可用的推荐系统通常需要完成数据导入、数据存储、推荐计算、接口服务、缓存加速、前端展示、指标评估和部署运维等多个环节。如果只在 Notebook 中计算推荐结果,虽然可以验证算法公式,但难以体现推荐系统在线服务的真实流程。因此,本课程设计选择将电影推荐算法与 Web 服务、数据库、缓存和容器化部署结合起来,构建一个能够运行、交互和评估的云端电影推荐系统原型。推荐系统评估研究也指出,除推荐准确率外,排序质量、响应性能和实验可复现性同样会影响系统效果判断^[8]。

本课程设计以 FastAPI Movie Recommender 项目为基础,将原有单体 Demo 扩展为一个基于 Docker Compose 部署的电影推荐系统。系统采用 FastAPI 提供后端接口,使用 Streamlit 展示交互页面,使用 PostgreSQL 保存 MovieLens 数据,使用 Redis 缓存推荐结果,并提供功能测试和抽样离线评估能力。该设计既关注推荐算法的基本效果,也关注系统在真实 Web 服务中的可部署性、可维护性和可扩展性。

1.2 课题意义

本课题围绕电影推荐系统展开,既具有一定的应用价值,也具有明确的课程实践意义。通过该系统,可以把推荐算法、后端开发、数据库设计、缓存机制、前端展示和容器化部署联系起来,形成一个相对完整的工程闭环。系统能够根据用户选择的电影返回相似电影,为电影内

容检索、兴趣发现和个性化推荐提供辅助；同时，推荐结果中包含相似度得分和推荐原因，便于观察协同过滤思想在具体场景中的应用过程。

从课程设计角度看，本课题的价值不只在实现一个推荐算法，而在于把算法放入真实系统环境中进行验证。MovieLens 数据集提供了用户-电影评分交互数据，PostgreSQL 和 Redis 分别承担数据持久化与缓存加速，FastAPI 和 Streamlit 分别承担接口服务与页面展示，Docker Compose 则负责多服务统一编排。通过这一过程，可以更完整地理解推荐系统从数据处理、推荐计算到在线展示和实验评估的实现链路。

1.3 课程设计目标

本课程设计的总体目标是完成一个可以运行、可以交互、可以部署、可以初步评估的电影推荐系统原型。系统不仅需要能够给出推荐结果，还需要具备清晰的数据链路、接口设计、页面展示和实验评价方法。围绕这一总体目标，本文主要完成以下任务：

1. 完成 MovieLens 数据集的导入与管理，建立电影、用户、评分、点击事件和实验结果等基础数据表，为推荐计算和结果分析提供数据支撑。
2. 设计并实现电影检索、相似电影推荐、推荐解释和 Demo 点击统计等核心功能，使用户能够在前端页面中完成完整的推荐交互流程。
3. 使用 FastAPI、Streamlit、PostgreSQL、Redis 和 Docker Compose 搭建可部署的推荐系统，实现后端接口、前端展示、数据存储、缓存加速和多容器编排。
4. 设计功能测试与抽样离线评估方案，从推荐命中情况、排序效果和响应延迟等角度分析系统表现，并总结后续可改进的方向。

需求分析

2.1 需求概述

本课程设计面向电影推荐这一典型应用场景，目标是构建一个能够完成电影检索、相似电影推荐、结果展示和初步评估的推荐系统原型。系统的核心需求不是单独验证某个算法公式，而是围绕用户使用流程，将数据导入、推荐计算、接口服务、前端交互、缓存优化和评估分析连接起来，形成一个完整的课程设计系统。

从用户角度看，系统应当能够帮助用户根据一部已知电影发现相似影片；从系统角度看，后端需要能够稳定地读取电影和评分数据，计算推荐结果，并将结果以接口形式提供给前端；从课程设计角度看，系统还需要能够展示推荐依据和实验指标，便于说明设计思路与实现效果。

2.2 用户角色与使用场景

本系统主要面向两类使用角色：

1. 普通用户：通过前端页面搜索电影，选择感兴趣的电影，并查看系统返回的相似电影推荐结果。
2. 系统设计与评估人员：通过接口和 Evaluation 页面观察推荐效果、响应延迟、点击统计和系统运行状态。

典型使用流程如下：

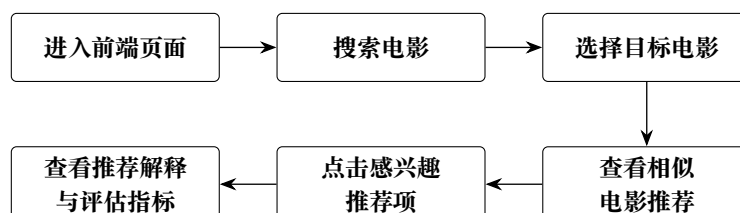


Figure 2-1 用户典型使用流程

该流程覆盖了电影推荐系统中从用户输入、后端计算到结果反馈的主要环节，也为后续改进推荐算法和缓存策略提供了基础。

2.3 功能需求

根据上述使用场景，系统需要满足以下功能需求：

功能模块	需求说明
电影检索	支持用户按照关键词搜索电影，并展示电影标题与类型信息
相似推荐	用户选择一部电影后，系统返回 Top-K 相似电影列表
推荐解释	每条推荐结果展示相似度得分和推荐原因
点击统计	记录本地 Demo 中推荐结果的曝光和点击，用于展示 Demo CTR
电影新增	支持演示场景中新增电影，用于模拟冷启动问题
离线评估	提供 Precision@K、Recall@K、NDCG@K、HitRate@K 和延迟指标

其中，电影检索和相似推荐是系统的核心功能；推荐解释和点击统计用于提高系统的可观察性；电影新增功能用于展示推荐系统在冷启动场景下可能遇到的问题；离线评估功能则用于从指标角度分析当前推荐方法的效果。

2.4 数据需求

推荐系统依赖用户与电影之间的交互数据，因此数据需求是本系统的重要组成部分。系统需要至少包含电影信息、用户信息和评分信息三类基础数据。电影信息用于展示标题和类型，用户评分信息用于计算电影之间的相似度，点击事件和评估结果则用于记录系统运行过程中的反馈与实验数据。

本系统选择 MovieLens 系列数据集作为实验数据来源。MovieLens 数据集由 GroupLens 项目长期维护，在推荐系统研究中使用较广，具有数据格式规范、评分行为明确、便于复现实验等特点。项目中支持 MovieLens 1M、MovieLens 32M 和 MovieLens Latest Full 等不同规模的数据，其中 MovieLens 1M 适合开发调试，MovieLens 32M 更适合作为相对稳定的课程设计和实验主数据集，Latest Full 则适合用于体验较新的数据和进行系统压力测试。

从课程设计角度看，不同规模的数据集承担的作用并不相同。小规模数据集便于快速验证导入脚本、数据库模型和推荐接口是否正确；中等及较大规模数据集能够更真实地暴露查询延迟、索引设计和缓存命中率等工程问题；持续更新的数据集则适合观察系统在数据增量变化时的可维护性。因此，本系统没有只使用一个最小样例数据集，而是按照开发、实验和扩展三类场景组织数据来源。

数据集版本	典型规模	在本项目中的作用
MovieLens 1M	约 100 万条评分	用于本地快速调试导入流程、表结构和推荐接口，运行速度快，适合排查程序错误
MovieLens 32M	约 3200 万条评分	作为主要实验数据集，用于支撑相似电影推荐、离线评估、缓存测试和接口性能观察
MovieLens Latest Full	持续更新的完整数据	用于后续扩展和压力测试，可验证系统面对更大规模或更新数据时的适应能力

MovieLens 现代 CSV 格式主要包含以下文件：

文件	主要字段	用途
movies.csv	movieId, title, genres	保存电影标题和类型信息
ratings.csv	userId, movieId, rating, timestamp	保存用户对电影的评分行为
links.csv	movieId, imdbId, tmdbId	保存电影与外部电影网站的编号映射
tags.csv	userId, movieId, tag, timestamp	保存用户为电影添加的文本标签

当前系统主推荐链路主要使用 movies.csv 和 ratings.csv，links.csv 可用于后续扩展外部电影信息，tags.csv 可用于后续内容特征增强或混合推荐。MovieLens 32M 没有单独的 users.dat 人口统计文件，系统会根据 ratings.csv 中出现的不同 userId 自动生成用户表。

当前已导入数据库的主要数据规模如下：

数据表	数量
users	200,948
movies	87,618
ratings	32,921,437

从规模上看，ratings 表达到 3200 万级，远大于教学中常见的 MovieLens 100K 或 MovieLens 1M 数据集。这使得系统不能只依赖内存中的临时数据结构完成所有计算，而需要借助 PostgreSQL 进行持久化存储，并结合 Redis 缓存减少重复推荐请求带来的计算开销。对于本课程设计而言，这一数据规模也能更好体现数据库索引、缓存和服务化推荐接口的必要性。

2.5 非功能需求

除基本业务功能外，系统还需要满足一定的工程化要求：

1. **可部署性**：系统应通过 Docker Compose 启动前端、后端、数据库和缓存服务，降低环境配置复杂度。
2. **可维护性**：项目目录应清晰区分 backend、frontend、trainer、evaluator、spark 和 docs 等模块。
3. **可扩展性**：推荐服务应保留 ALS、BPR、Hybrid、RecBole 和 Spark ALS 等后续算法与实验扩展入口。
4. **可观测性**：后端需要提供健康检查、缓存指标、系统指标和评估指标，便于观察系统运行状态。
5. **响应性能**：在线推荐需要尽量降低响应延迟，并通过索引、缓存或离线预计算方式缓解大规模评分数据带来的性能压力。

2.6 开发与运行环境

为满足上述需求，本系统采用如下技术环境：

项目	配置
后端服务	FastAPI
前端展示	Streamlit
数据库	PostgreSQL
缓存	Redis
容器编排	Docker Compose
数据集	MovieLens 32M / Latest CSV 格式

总体设计

3.1 系统总体架构

系统采用前后端分离和容器化部署方式，在线服务链路如下：

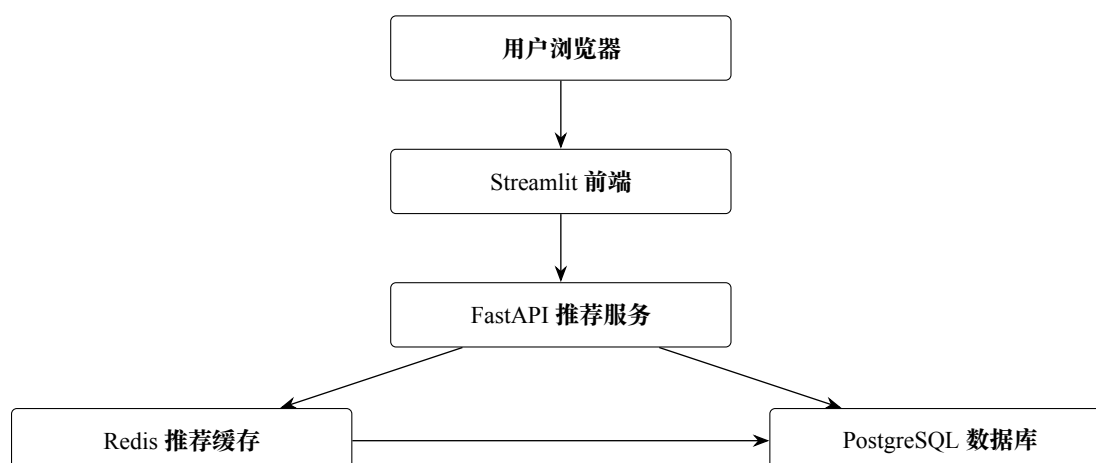


Figure 3-2 系统总体架构

其中，Streamlit 负责交互页面展示，FastAPI 负责提供 REST API，PostgreSQL 负责保存电影、用户、评分和点击事件数据，Redis 用于缓存推荐结果和降低重复计算开销。该架构将展示层、服务层、数据层和缓存层解耦，便于后续替换推荐算法或扩展部署规模。

3.2 系统模块划分

系统主要划分为以下模块：

模块	主要职责
backend	FastAPI 接口、数据库模型、推荐逻辑和系统指标
frontend	Streamlit 页面、电影搜索、推荐结果展示和评估页面
trainer	后续 ALS/BPR 等离线模型训练脚本
evaluator	抽样评估、RecBole 配置和实验结果记录
spark	Spark 预处理与 Spark MLlib ALS 对照实验入口
docs	课程设计报告、实验记录和系统说明文档

3.3 数据集与数据库设计

当前系统已经导入 MovieLens 现代 CSV 数据集。数据导入流程首先读取 movies.csv 和 ratings.csv，将电影信息和评分行为写入 PostgreSQL；如果目录中存在 links.csv 和 tags.csv，系统也可以将外部编号映射和用户标签导入对应扩展表。导入完成后，推荐服务主要围绕电影表和评分表进行查询与相似度计算。

数据库统计如下：

表	数量
users	200,948
movies	87,618
ratings	32,921,437

数据库核心表包括 users、movies、ratings、user_events、recommendation_cache 和 experiment_results。其中，movies 保存电影标题与类型，ratings 保存用户评分行为，user_events 用于记录前端点击事件，recommendation_cache 用于保存推荐缓存，experiment_results 用于记录评估指标。

各类数据在系统中的作用如下：

数据类型	系统作用
电影基础信息	支持电影搜索、标题展示、类型展示和新增电影演示
用户评分数据	支持电影相似度计算、协同过滤推荐和抽样离线评估
用户点击事件	记录 Demo 曝光与点击行为，用于计算本地 Demo CTR
推荐缓存数据	缓存重复推荐请求结果，降低在线计算开销
实验结果数据	保存 Precision@K、Recall@K、NDCG@K 和延迟等指标

与 MovieLens 1M 相比，当前数据规模更大，能够更好体现数据库、缓存和服务化部署的必要性。但由于 MovieLens Latest 数据集会随时间更新，因此正式固定实验结果更适合使用 MovieLens 32M 作为稳定 benchmark。

详细设计与系统实现

4.1 电影相似推荐设计

当前系统主流程采用电影到电影的相似推荐方式。用户输入或选择一部电影后，系统基于评分行为计算候选电影与输入电影的相似度，并返回 Top-K 推荐结果。

为保证在 3200 万级评分数据上仍能在线交互，当前实现采用抽样评分向量余弦相似度：

1. 获取输入电影的一部分评分用户；
2. 获取这些用户评分过的其他电影；
3. 在采样用户空间中构造电影评分向量；
4. 计算候选电影与输入电影之间的余弦相似度；
5. 返回 Top-K 相似电影。

推荐解释示例：

rating cosine: 124 sampled shared users

前端将其解释为“相似评分行为”，表示这些电影在共同评分用户上的评分模式较接近。该方法实现简单、解释清晰，适合作为课程设计阶段的基础推荐算法。

4.2 接口与缓存设计

后端主要提供以下接口：

接口	功能
GET /movies/search	根据关键词搜索电影
GET /recommend/movie	根据电影 ID 返回相似电影
POST /clicks	记录推荐结果点击事件
POST /movies	新增电影数据
GET /health	返回服务健康状态
GET /metrics/cache	返回缓存相关指标

Redis 主要用于缓存相同输入电影和相同 Top-K 参数下的推荐结果。对于重复访问的热门电影，缓存能够减少数据库查询和相似度重复计算，从而降低接口响应时间。

4.3 点击率与冷启动设计

系统记录本地 Demo 中的推荐曝光和点击：

Demo CTR = clicks / recommendations_shown

该指标只表示本地演示点击率，不代表真实线上用户点击率。因此报告中使用“Demo CTR”命名，避免与生产环境 CTR 混淆。

用户还可以在侧边栏新增电影，例如输入“战狼”、类型为 Action|Drama、初始评分为 3.0。由于 MovieLens 原始数据中不存在《战狼》，该场景可解释为模拟新电影加入系统后的冷启动推荐。对于缺少评分历史的新电影，系统无法直接计算可靠的协同过滤相似度，因此只能使用类型、标题和初始评分作为辅助信息。

4.4 系统实现流程

系统实现流程包括数据导入、服务启动、前端交互和推荐评估四个环节：

1. 将 MovieLens 数据集导入 PostgreSQL，生成 movies、users 和 ratings 等核心表。
2. 使用 Docker Compose 启动 PostgreSQL、Redis、FastAPI 后端和 Streamlit 前端。
3. 用户在前端搜索电影并选择 seed movie，后端根据评分行为计算相似电影。
4. 前端展示推荐电影、类型、相似度得分、推荐解释和 Demo CTR。
5. Evaluation 页面调用抽样评估接口，展示推荐指标和响应延迟。

4.5 功能测试方案

功能测试用于验证系统是否达到课程设计的基本功能要求：

功能	验收方式
电影搜索	输入 Toy Story 能检索到 Toy Story (1995)
相似推荐	GET /recommend/movie?movie_id=1&limit=4 返回 4 条推荐
推荐解释	每条推荐显示相似原因和 Score
点击统计	点击推荐项后 Demo CTR 发生变化
新增电影	侧边栏新增电影后写入数据库

4.6 离线评估方案

系统提供轻量级 leave-one-out 抽样评估：

对每个抽样用户：

1. 取一部高评分电影作为 seed movie；
2. 取另一部高评分电影作为 target movie；
3. 用 seed movie 生成 Top-K 推荐；
4. 如果 target movie 出现在 Top-K 中，则记为命中。

评估指标包括：

指标	含义
Precision@K	Top-K 推荐中命中目标的比例
Recall@K	目标电影是否被召回
NDCG@K	命中位置越靠前得分越高
HitRate@K	样本用户中至少命中一次的比例
Avg Latency	平均推荐响应时间
p95 Latency	95 分位响应时间

测试结果与分析

5.1 功能测试结果

以 Toy Story (1995) 为输入电影，系统能够返回基于评分行为的相似电影推荐。一次接口结果示例如下：

推荐电影	类型	Score	推荐原因
Matrix, The (1999)	Action Sci-Fi Thriller	0.3412	rating cosine: 124 sampled shared users
Shawshank Redemption, The (1994)	Drama	0.3401	rating cosine: 122 sampled shared users
Star Wars: Episode IV (1977)	Action Adventure Fantasy Sci-Fi	0.3380	rating cosine: 124 sampled shared users
Star Wars: Episode V (1980)	Action Adventure Drama Sci-Fi War	0.3369	rating cosine: 123 sampled shared users

结果说明系统已经能够达到核心功能：根据输入电影返回相似电影，并且具备可解释推荐结果。

5.2 抽样评估结果

当前一次抽样设置：

```
sample_size = 10
k = 10
model = movie-cosine
```

实验结果如下：

指标	数值
Precision@10	0.0100
Recall@10	0.1000
NDCG@10	0.0500
HitRate@10	0.1000
Avg Latency	1869.201 ms
p95 Latency	9912.077 ms

5.3 结果分析

从推荐效果看，当前抽样 HitRate@10 为 0.1，说明在 10 个抽样用户中有 1 个用户的隐藏目标电影被 Top-10 推荐命中。该指标不高，主要原因包括：

1. 当前方法是在线抽样版电影相似度计算，不是完整离线相似矩阵。
2. 抽样 leave-one-out 只保留一个目标电影，评价较严格。
3. MovieLens 32M 电影数量达到 87,618，候选空间较大。
4. 未训练 ALS/BPR 等矩阵分解模型，个性化能力有限。

从系统性能看，平均延迟约 1.87 秒，p95 延迟接近 9.91 秒，说明当前在线按需计算在大规模数据上存在明显性能瓶颈。这一结果也证明系统需要 Redis 缓存、离线预计算和模型服务化。

存在问题、改进方向与总结

6.1 热门电影偏置

从 Demo 推荐结果看, Matrix、Shawshank Redemption、Star Wars 等热门电影容易出现在结果前列。这说明评分共现和采样相似度容易受到热门电影影响。后续可通过以下方式改进:

1. 对高热电影进行流行度惩罚。
2. 使用调整余弦相似度或 Pearson 相似度。
3. 引入 BM25/TF-IDF 式的用户共现降权。
4. 使用 ALS/BPR 学习潜在向量, 降低单纯共现带来的热门偏置。

6.2 响应延迟

当前相似度在线计算依赖 PostgreSQL 查询和 Python 聚合, 在 32M 评分规模下 p95 延迟偏高。后续优化方向:

1. 离线预计算 Top-N 相似电影表。
2. 将相似度结果写入 Redis 或 PostgreSQL 物化表。
3. 对 ratings(movie_id, user_id) 和 ratings(user_id, movie_id) 建立合适索引。
4. 使用 implicit 模型训练 item factors, 在线只做向量近邻检索。

6.3 实验完整性

当前 Evaluation 页面已经形成基本评估闭环, 但仍属于抽样评估。若要达到更完整的课程设计或论文实验标准, 需要补充:

1. Popular、Item-CF、ALS、BPR、Hybrid 的横向对比。
2. RecBole 离线评估结果。
3. Spark ALS 分布式训练对照。
4. 缓存命中前后接口响应时间对比。
5. Docker CPU/内存占用记录。

6.4 课程设计总结

本课程设计完成了一个基于 Docker Compose、FastAPI、Streamlit、PostgreSQL 和 Redis 的电影推荐系统原型。系统能够支持电影搜索、相似电影推荐、推荐解释、相似度得分、本地 Demo CTR 和抽样评估指标展示。

与原始 FastAPI Movie Recommender Demo 相比, 当前系统在以下方面进行了扩展:

1. 数据库从 SQLite 扩展到 PostgreSQL。

2. 数据规模从 MovieLens 100K/1M 扩展到 32M 级别。
3. 增加 Redis 缓存和容器化多服务部署。
4. 增加 Evaluation 页面和抽样离线指标。
5. 增加冷启动电影添加入口和 Demo CTR 展示。

从课程设计角度看，本系统不仅实现了推荐算法的基本流程，也完成了一个可交互、可部署、可评估的 Web 应用闭环。后续工作可以继续围绕离线预计算、ALS/BPR 模型训练、RecBole 评估、Spark ALS 对照实验和 Nginx 统一入口部署进行完善，使系统从课程设计原型进一步发展为更完整的推荐系统实验平台。

参考文献

- [1] Adomavicius G, Tuzhilin A. Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions[J]. IEEE Transactions on Knowledge and Data Engineering, 2005, 17(6): 734-749.
- [2] Linden G, Smith B, York J. Amazon.com recommendations: Item-to-item collaborative filtering [J]. IEEE Internet Computing, 2003, 7(1): 76-80.
- [3] Harper F M, Konstan J A. The MovieLens datasets: History and context[J]. ACM Transactions on Interactive Intelligent Systems, 2015, 5(4): 1-19.
- [4] Sarwar B, Karypis G, Konstan J, et al. Item-based collaborative filtering recommendation algorithms[C]. in: Proceedings of the 10th International Conference on World Wide Web. 2001: 285-295.
- [5] Koren Y, Bell R, Volinsky C. Matrix factorization techniques for recommender systems[J]. Computer, 2009, 42(8): 30-37.
- [6] Hu Y, Koren Y, Volinsky C. Collaborative filtering for implicit feedback datasets[C]. in: 2008 Eighth IEEE International Conference on Data Mining. 2008: 263-272.
- [7] Rendle S, Freudenthaler C, Gantner Z, et al. BPR: Bayesian personalized ranking from implicit feedback[C]. in: Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence. 2009: 452-461.
- [8] Herlocker J L, Konstan J A, Terveen L G, et al. Evaluating collaborative filtering recommender systems[J]. ACM Transactions on Information Systems, 2004, 22(1): 5-53.